

Brief Papers

Tree Ensembles on the Induced Discrete Space

Olca Taner Yıldız

Abstract—Decision trees are widely used predictive models in machine learning. Recently, K -tree is proposed, where the original discrete feature space is expanded by generating all orderings of values of k discrete attributes and these orderings are used as the new attributes in decision tree induction. Although K -tree performs significantly better than the proper one, their exponential time complexity can prohibit their use. In this brief, we propose K -forest, an extension of random forest, where a subset of features is selected randomly from the induced discrete space. Simulation results on 17 data sets show that the novel ensemble classifier has significantly lower error rate compared with the random forest based on the original feature space.

Index Terms—Classification, decision trees, feature extraction, random forest.

I. INTRODUCTION

A decision tree is a hierarchical structure made up of internal decision nodes and terminal leaves, which carry the label of one of K classes. The input vector is composed of d attributes, $\mathbf{x} = [x_1, \dots, x_d]$. Each decision node m implements function $F_m(\mathbf{x})$ and depending on the outcome of the function F_m , the input is forwarded to one of the child nodes. In a univariate decision tree [4], the function F_m uses only one attribute, i.e., one dimension of $\mathbf{x}$, x_j . If that attribute is discrete, there will be L children (branches) of each internal node corresponding to the L different outcomes of the decision. Given an input to classify, starting from the root node, one applies the function at each internal node and the input is forwarded to one of the L branches depending on the outcome. This process is repeated recursively until a leaf node is hit at which point the class label of the leaf constitutes the output.

One of the drawbacks of the L -ary splits is that the training examples are separated into small subsets, which in turn gives us a poor predictor for the unseen test instances. One can convert discrete features having $L > 2$ different values to L binary features using 1-of- L encoding, this will result in a larger tree than the former. Although there are alternative approaches to handle the selection bias that favors the attributes having many values over those with a few values [5], [6], those approaches cannot help if the attributes have nearly similar number of values.

Another drawback of using discrete attributes in the form of $x_i = v_{ij}$ is that there is only a single possible split for each discrete attribute. On the other hand, there are $n - 1$ different possible splits for a continuous attribute, where n represents the number of distinct values of that continuous attribute. Yıldız [1] proposed the feature extraction from discrete space and its application to univariate tree induction, i.e., K -trees. To increase the number of distinct splits for data sets with discrete attributes, we defined k -ordering, where for each subset (size k) of the attributes, we combine them by generating all possible orderings of the values of those attributes exhaustively. After exploding the feature space, we can apply the usual decision

tree algorithm using these orderings as the new attributes. In this way, the number of distinct possible splits for each extracted feature will be $n_1 \times n_2 \times \dots \times n_k$, where n_i represents the number of distinct values of the selected feature i .

Although K -trees are significantly better than their counterparts, the time complexity in the training phase (due to the exhaustive search nature of decision trees in the induced space) prevents them from being a good competitor. In this brief, we propose an extension to K -tree, namely, K -forest. Instead of searching for the best split on the whole induced space, we search for the best split on a significantly smaller and therefore tractable subset of the induced space.

This brief is organized as follows. We review the definition of k -ordering and its application to univariate decision tree induction in Sections II and III, respectively. We discuss K -forest in Section IV. We give experimental results in Section V. Finally, the conclusion is drawn in Section VI.

II. k -ORDERING

A. Definition

Let $x_1, x_2, \dots, x_d$ be d discrete attributes of a data set D . Each attribute x_i can have n_i distinct values which can be represented as $v_{i1}, v_{i2}, \dots, v_{in_i}$.

Definition 1: Given an ordered set of k attributes $\{x_{s_1}, x_{s_2}, \dots, x_{s_k}\}$ from a d -dimensional data set D , k -ordering is defined as the permutation list $(p_1, p_2, \dots, p_k)$, where p_i is a permutation of values of the feature x_{s_i} (a permutation of $v_{s_i1}, v_{s_i2}, \dots, v_{s_in_{s_i}}$).

For example, [(female, male), (false, true), (cold, mild, hot)] is a three-ordering, where selected $k = 3$ features have two, two, and three distinct values, respectively. Note that we can use both categorical discrete attributes (such as female, male) as well as nominal discrete attributes (such as small, medium, and large) to define the k -ordering. There are

$$\sum_{s_1, s_2, \dots, s_k \in \{1, 2, \dots, d\}} n_{s_1}! n_{s_2}! \dots n_{s_k}!$$

distinct k -orderings of a d -dimensional data set. As an example, given a data set with two dimensions having values (cold, mild, hot) and (female, male), there are $8 (3! + 2!)$ and $24 (3! 2! + 2! 3!)$ distinct 1 and two-orderings, respectively.

B. Ordering Relations Based on k -Ordering

Continuing the definition of k -ordering, we can produce relational operators that compare two instances. Formally, given a k -ordering of a data set D and two instances $\mathbf{x}^{(i)} = [x_1^{(i)}, \dots, x_d^{(i)}]$ and $\mathbf{x}^{(j)} = [x_1^{(j)}, \dots, x_d^{(j)}]$ from this data set.

Definition 2: $[x_{s_1}^{(i)}, \dots, x_{s_k}^{(i)}] < [x_{s_1}^{(j)}, \dots, x_{s_k}^{(j)}]$ if and only if $x_{s_r}^{(i)} = x_{s_r}^{(j)}$ for $r = 1, \dots, t \geq 0$, and $x_{s_{t+1}}^{(i)}$ comes before $x_{s_{t+1}}^{(j)}$ in permutation p_{t+1} .

Definition 3: $[x_{s_1}^{(i)}, \dots, x_{s_k}^{(i)}] > [x_{s_1}^{(j)}, \dots, x_{s_k}^{(j)}]$ if and only if $x_{s_r}^{(i)} = x_{s_r}^{(j)}$ for $r = 1, \dots, t \geq 0$, and $x_{s_{t+1}}^{(i)}$ comes after $x_{s_{t+1}}^{(j)}$ in permutation p_{t+1} .

Manuscript received August 11, 2013; revised October 20, 2014, February 18, 2015, and April 24, 2015; accepted May 2, 2015. Date of publication May 21, 2016; date of current version April 15, 2016.

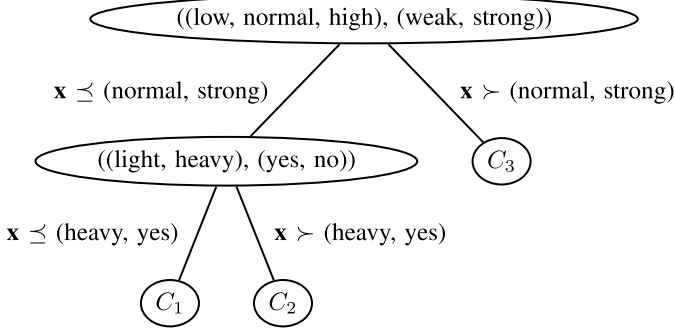
The author is with the Department of Computer Engineering, Işık University, Istanbul, Istanbul 34398, Turkey (e-mail: olcaytaner@isikun.edu.tr).

Digital Object Identifier 10.1109/TNNLS.2015.2430277

```

Set AttributeSet( $k$ )
1  $S = \{\}$ 
2 for each attribute permutation  $(s_1, \dots, s_k)$ 
3   foreach permutation  $p_1$  of values of  $x_{s_1}$ 
4     ...
5   foreach permutation  $p_k$  of values of  $x_{s_k}$ 
6      $S = S \cup \{(p_1, \dots, p_k)\}$ 
7 return  $S$ 

```

Fig. 1. Pseudocode of the exhaustive k -ordered attribute search algorithm.Fig. 2. Example K -tree ($k = 2$).

Definition 4: $[x_{s_1}^{(i)}, \dots, x_{s_k}^{(i)}] = [x_{s_1}^{(j)}, \dots, x_{s_k}^{(j)}]$ if and only if $x_{s_r}^{(i)} = x_{s_r}^{(j)}$ for all $r = 1, \dots, k$.

For example, given the two-ordering $[(\text{low}, \text{normal}, \text{high}), (\text{weak}, \text{strong})]$, all possible values of the instances can be sorted as $(\text{low}, \text{weak}) < (\text{low}, \text{strong}) < (\text{normal}, \text{weak}) < (\text{normal}, \text{strong}) < (\text{high}, \text{weak}) < (\text{high}, \text{strong})$.

C. Splits Based on k -Ordering

If the instances of a data set can be sorted based on a k -ordering, we can also list all possible splits based on that k -ordering. For example, given the two-ordering $[(\text{low}, \text{normal}, \text{high}), (\text{weak}, \text{strong})]$, all possible splits are: 1) $\mathbf{x} \leq (\text{low}, \text{weak})$; 2) $\mathbf{x} \leq (\text{low}, \text{strong})$; 3) $\mathbf{x} \leq (\text{normal}, \text{weak})$; 4) $\mathbf{x} \leq (\text{normal}, \text{strong})$; and 5) $\mathbf{x} \leq (\text{high}, \text{weak})$.

More formally, since each k -ordering defines a new attribute

$$\sum_{s_1, s_2, \dots, s_k \in 1, 2, \dots, d} n_{s_1}! n_{s_2}! \dots n_{s_k}! \quad (1)$$

new attributes are generated. For each new attribute, we can use one of the $n_{s_1} \times n_{s_2} \times \dots \times n_{s_k} - 1$ distinct splits in a tree node.

The pseudocode for finding the exhaustive set of k -ordered attributes is shown in Fig. 1. First, we initialize the result set S (Line 1). For each k -permutation of the attributes (Line 2), we extract all possible k -orderings by traversing the attributes iteratively (Lines 3–5). Each possible k -ordering and corresponding permutation $(p_1, \dots, p_k)$ is added to S (Line 6).

III. APPLICATION TO DECISION TREE INDUCTION

A. K -Tree

In this section, we apply k -ordering to find k -ordered splits in the univariate decision tree algorithm ID3 [5]. The idea is as follows: at each decision node, we generate all possible k -orderings using the algorithm in Fig. 1 and all split points as explained in Section II-C. For all attributes and for all split points of those attributes, we calculate impurity and choose the split point and k -ordering that has the minimum entropy.

Fig. 2 shows an example K -tree with two decision and three leaf nodes. In the root node, the best ordering and the best split

```

Impurity( $sp, r, \mathcal{X}$ )
1 for  $i = 1$  to  $K$ 
2    $N_i^L = 0, N_i^R = 0$ 
3 for  $i = 1$  to  $\mathcal{X}.\text{size}$ 
4    $\mathbf{x} = \mathcal{X}[i]$ 
5    $j = \text{class of } \mathbf{x}$ 
6   if  $\mathbf{x}$  satisfies  $sp$  according to ordering  $r$ 
7      $N_j^L = N_j^L + 1$ 
8   else
9      $N_j^R = N_j^R + 1$ 
10 return impurity calculated from  $N_i^L$  and  $N_i^R$ .

```

Fig. 3. Pseudocode of the algorithm that finds the impurity of a k -ordered split sp of a k -ordering r for an instance list $\mathcal{X}$.

for that ordering are $[(\text{low}, \text{normal}, \text{high}), (\text{weak}, \text{strong})]$ and $\mathbf{x} \leq (\text{normal}, \text{strong})$, respectively. This way, K -trees can effectively solve the replication problem of trees. Replication problem can also occur when the data contains attributes with high arity values, i.e., attributes with large number of possible values. If a tree has high arity attributes (say $L \geq 5$) then it will quickly fragment the data in that node into small partitions. One possible solution for this problem is constructing the subsets of the attribute values [7].

B. Impurity Calculation

Fig. 3 shows the pseudocode that finds the impurity of a k -ordered split for a given k -ordering r . First, we initialize the counts of the left and right branches (Line 2). For each instance $\mathbf{x}$ in the instance list $\mathcal{X}$, we compare its attribute values with the split point sp according to the current k -ordering (Line 6). If $\mathbf{x}$ satisfies the split sp according to the k -ordering r , we update counts of the left branch (Line 7); otherwise, we update counts of the right branch (Line 9). Using the class counts of the left and right branches, we can calculate the impurity using entropy [8], Gini index [9], or any other metric (Line 10). In our experiments, we use entropy to calculate impurity. Our experiments and those of the previous researchers indicate that there is usually no significant difference between these impurity measures.

IV. K -FOREST

The concept of bootstrap aggregating, namely bagging, was introduced in [10]. The random forest method improves bagging idea with randomizing features at each decision node [2] and called these random decision trees as weak learners. In the prediction time, these weak learners are combined using committee-based procedures.

The principal idea in bagging is to average many noisy but approximately unbiased models, in other words, decrease the variance. Trees are ideal candidates for bagging, they are inherently hierarchical (or complex) models, and if they are grown enough, have low bias. Trees are also intrinsically noisy; therefore they benefit greatly from averaging. In addition to these, since each tree is identically distributed, the expectation of an average of these trees is the same as the expectation of any of them. Therefore, the bias of bagged trees is the same of individual trees, and the only way to decrease the overall error is through variance reduction.

The idea in random forests is to reduce variance of bagging by reducing the correlation between the trees, without increasing the variance too much. This is accomplished in the tree-growing process via random selection of input attributes. In the testing phase, a random forest obtains a class vote from each tree, and then classifies using majority vote.

Fig. 4 shows the pseudocode of the algorithm weak- K -tree that creates the weak learner for an instance list $\mathcal{X}$. In this case,

```

Weak- $K$ -Tree( $\mathcal{X}, k, f$ )
1  bestImpurity =  $\infty$ 
2   $S = \text{AttributeSet}(k)$ 
3  Randomly select  $f$  attributes  $F$  from  $S$ 
4  for  $i = 1$  to  $f$ 
5      Search for the best split  $sp$  on attribute  $F[i]$ 
6       $\tau = \text{Impurity}(sp, F[i], \mathcal{X})$ 
7      if  $\tau < \text{bestImpurity}$ 
8          bestImpurity =  $\tau$ 
9          bestSplit =  $sp$ 
10 Divide  $\mathcal{X}$  into  $\mathcal{X}^L$  and  $\mathcal{X}^R$  according to bestSplit
11 Weak- $K$ -Tree( $\mathcal{X}^L, k, f$ )
12 Weak- $K$ -Tree( $\mathcal{X}^R, k, f$ )

```

Fig. 4. Pseudocode of the algorithm that creates the weak learner for an instance list $\mathcal{X}$ using f random attributes at each decision node.

```

 $K$ -Forest( $\mathcal{X}, k, f, B$ )
1  for  $i = 1$  to  $B$ 
2      Build bootstrap sample  $\mathcal{D}_i$  from sample  $\mathcal{X}$ 
3       $\mathcal{T}_i = \text{Weak-}K\text{-Tree}(\mathcal{D}_i, k, f)$ 
4  Return prediction by aggregating classifiers  $\mathcal{T}_i$ 

```

Fig. 5. Pseudocode of the algorithm that creates the K -forest consisting of B weak K -trees for an instance list $\mathcal{X}$.

the weak learner is not the original decision tree, but the modified K -tree, where at each decision node we choose f attributes from the expanded space randomly (Line 3). For each of these f attributes, the best split is found by searching all possible split points like K -tree exhaustively (Line 5), and the overall best split from these f best splits is chosen according to impurity (Lines 7–9). Tree construction continues recursively with left and right children (Lines 11–12).

Fig. 5 shows the pseudocode of the algorithm K -forest that creates B weak K -trees for an instance list $\mathcal{X}$. First, we build B -independent bootstrap samples $\mathcal{D}_1, \dots, \mathcal{D}_B$ by drawing with replacement among the original training sample $\mathcal{X}$ (Line 2). For each bootstrap sample, we run the weak- K -tree algorithm yielding the K -tree $\mathcal{T}_i$ (Line 3). As a last step, we combine the predictions of weak learners using a committee-based procedure such as majority vote.

When the number of attributes is large, but the fraction of relevant attributes is small, random forests perform poorly with small f . At each split of the decision tree, the chance can be small that the relevant attributes will be selected. The number of distinct attributes of K -forests is significantly large compared with original random forests. Therefore, if f increases, we expect significant performance increase in K -forests.

The main problem of K -tree is its time complexity. In the K -tree algorithm, at each decision node, we search for the best split over the whole induced discrete space, the set S ($|S| = \sum_{s_1, s_2, \dots, s_k \in \{1, 2, \dots, d\}} n_{s_1}! n_{s_2}! \dots n_{s_k}!$). For each attribute in that space S , searching for the best split requires a single pass over n instances arriving at that node. Therefore, for each decision node, the time complexity of finding the best split is $\mathcal{O}(|S|n)$. If there are N_1 nodes in the K -tree, time complexity of constructing K -tree is $\mathcal{O}(|S|N_1n)$.

In the K -forest algorithm, at each decision node, we search for the best split over a significantly smaller subset of S , namely, f of them.

TABLE I
DETAILS OF THE DATA SETS. d : NUMBER OF ATTRIBUTES, C : NUMBER OF CLASSES, N : SAMPLE SIZE, AND n/v : n ATTRIBUTES HAVE v DISTINCT VALUES

Dataset	d	C	N	n/v
ACceptors	88	2	3889	88/4
ARTificial	5	2	320	5/2
BALance	4	3	625	4/5
CAR	6	4	1728	3/3, 2/4, 1/5
DONors	13	2	6246	13/4
FLAGs	26	8	194	13/2, 2/3, 2/4, 1/5, 1/6, ..., 1/14
HAYesroth	4	3	160	1/3, 3/4
KRVskp	36	2	3196	35/2, 1/3
MONks	6	2	432	2/2, 3/3, 1/4
MUSHroom	22	2	8124	5/2, 1/3, 5/4, 1/5, 2/6, ..., 1/12
NURSery	8	5	12960	1/2, 4/3, 2/4, 1/5
POSToperative	8	3	90	2/2, 5/3, 1/4
PROmotors	57	2	106	57/4
SPEct	22	2	267	22/2
SPLice	60	3	3190	60/4
TICtactoe	9	2	958	9/3
VOTE	16	2	435	16/2

For each of those f attributes, searching for the best split requires a single pass over n instances arriving at that node. Therefore, for each decision node, the time complexity of finding the best split is $\mathcal{O}(fn)$. If there are N_2 nodes in each B weak K -trees of K -forest on the average, the time complexity of constructing K -forest is $\mathcal{O}(BfN_2n)$.

In the random forest algorithm, at each decision node, like K -forest, we search for the best split among f attributes. Similarly, if there are N_3 nodes in each weak learner of random forest on the average, the time complexity of constructing random forest is $\mathcal{O}(BfN_3n)$.

V. EXPERIMENTS

A. Setup

We use a total of 17 data sets where 14 of them are from University of California Irvine (UCI) repository [3], *artificial* is a synthetic data set and 2 are (*acceptors* and *donors*) bioinformatics data sets (see Table I).

The data set is resampled using tenfold cross validation to generate ten training and test folds, train_i , test_i , $i = 1, \dots, 10$. The data set is divided into ten equal sized subsamples and on each fold i , nine subsamples are combined to create the training fold train_i and the remaining subsample is the test fold test_i , which are used to estimate the expected error of the algorithms. train_i are used to train the random forest, K -tree and K -forest algorithms. At each fold, when tune the hyperparameters of the algorithm, we separate 20% of train_i aside and use as the validation set val_i . val_i are used as a pruning set in the K -tree algorithm. val_i are also used as a validation set to tune the random attribute size in both random forest and K -forest. We tried possible values for f from the set $\{2^0, 2^1, \dots, 2^6, 2^7\}$ and choose the best according to the expected error on the validation set.

We use k -fold paired t test for pairwise comparison with confidence level $\alpha = 0.05$ (see the Appendix for the details of k -fold paired t test). Before starting any comparison, first, we check if the normality assumption holds and for this, we use the skewness test of normality [11] to the results of all of our classifiers on all data sets. Indeed, we see that on 98% of the cases, the normality test does not reject. This shows that k -fold paired t test, which assumes normality in the results, is applicable in our experiments.

B. K -Forest Versus K -Tree

In this section, we compare the performance of our proposed random forest algorithm (K -forest) with K -tree proper in terms

TABLE II

AVERAGE AND STANDARD DEVIATIONS OF THE ERROR RATES OF ALGORITHMS K -TREE WITH $k = 1, 2, 3, 4$ AND K -FOREST WITH $k = 1, 2, 3, 4$, $B = 100$. STATISTICALLY SIGNIFICANT DIFFERENCES ARE SHOWN IN BOLDFACE

Set	1-Forest	1-Tree	2-Forest	2-Tree	3-Forest	3-Tree	4-Forest	4-Tree
ACC	8.0 ± 0.7	12.2 ± 1.3	8.3 ± 1.1	N/A	8.5 ± 1.1	N/A	8.1 ± 0.9	N/A
ART	0.0 ± 0.0	0.9 ± 3.0	0.0 ± 0.0	0.9 ± 3.0	0.0 ± 0.0	0.9 ± 3.0	0.0 ± 0.0	0.0 ± 0.0
BAL	13.8 ± 2.4	23.1 ± 7.2	15.7 ± 3.7	22.3 ± 5.3	15.4 ± 2.7	N/A	13.6 ± 3.4	N/A
CAR	1.9 ± 0.9	4.0 ± 2.0	1.4 ± 1.1	2.0 ± 1.4	1.0 ± 0.7	2.9 ± 1.6	1.9 ± 0.8	N/A
DON	4.6 ± 0.6	6.0 ± 1.2	4.7 ± 0.8	6.2 ± 1.4	4.6 ± 0.8	N/A	4.6 ± 0.7	N/A
HAY	19.5 ± 9.9	16.5 ± 9.3	21.9 ± 9.4	20.3 ± 10.6	20.7 ± 7.6	23.2 ± 12.6	20.0 ± 5.7	N/A
KRV	0.4 ± 0.5	0.8 ± 0.6	0.7 ± 0.6	0.8 ± 0.4	0.4 ± 0.3	0.8 ± 0.5	0.6 ± 0.4	0.6 ± 0.4
MON	0.0 ± 0.0	9.5 ± 7.7	0.2 ± 0.8	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
NUR	0.3 ± 0.3	1.0 ± 0.4	0.1 ± 0.1	0.2 ± 0.1	0.0 ± 0.1	0.4 ± 0.3	0.0 ± 0.1	N/A
POS	30.1 ± 9.4	29.2 ± 6.9	30.3 ± 8.4	28.3 ± 4.7	35.2 ± 13.5	28.3 ± 4.7	31.5 ± 8.4	29.2 ± 6.9
PRO	14.0 ± 10.1	26.3 ± 9.6	5.7 ± 9.5	27.2 ± 14.2	8.5 ± 10.7	N/A	15.5 ± 12.7	N/A
SPE	17.5 ± 6.1	21.0 ± 1.6	18.7 ± 5.4	23.2 ± 4.4	20.9 ± 4.7	19.8 ± 2.1	21.3 ± 5.8	20.6 ± 1.7
SPL	3.9 ± 1.3	6.0 ± 1.9	3.8 ± 1.5	6.4 ± 1.7	4.3 ± 1.4	N/A	4.1 ± 1.4	N/A
TIC	1.4 ± 1.1	5.6 ± 2.3	1.4 ± 0.7	7.1 ± 2.9	1.3 ± 1.1	4.0 ± 2.6	1.3 ± 1.1	N/A
VOT	4.8 ± 3.3	4.4 ± 2.0	3.2 ± 2.4	4.4 ± 2.0	3.4 ± 2.9	4.4 ± 2.5	3.0 ± 2.4	4.3 ± 3.9

of generalization error and time complexity. We run our proposed algorithm for $k = 1, 2, 3, 4$, $B = 100$. We also run the K -tree algorithm for $k = 1, 2, 3, 4$, if time allowed.

Table II shows the average and standard deviations of the error rates of K -tree and K -forest algorithms. If the difference between K -tree and K -forest is statistically significant, we show the better one in boldface. If the time complexity of the K -tree algorithm for a given k is too large, we opted not to run that algorithm and showed the corresponding with N/A (not available). We see from the results that, K -forest is significantly better than K -tree in terms of error rate. 1-forest is significantly better than 1-tree in 10 data sets out of 15, 2-forest is significantly better than 2-tree in 7 data sets out of 14, 3-forest are significantly better than 3-tree in 4 data sets out of 10, and 4-forest is significantly better than 4-tree in 1 data set out of 6. Although the performance of K -forest with respect to K -tree diminishes as K increases, for many data sets, running K -tree with large K is not a possible option. Therefore, not only the performance of K -forest is better than K -tree, but it is also more useful for application purposes.

We also compare learning times of K -tree and K -forest algorithms. We cannot list the learning time of the algorithms due to lack of space. First of all, for two data sets, *mushroom* and *flags*, both the number of attributes and the number of distinct values of those attributes (v) are very large, and for this reason, running 1-tree is impractical. For all other data sets, 1-tree has significantly smaller time complexity than 1-forest. On the other hand, as K increases, the tables turn on, the time complexity of K -tree increases exponentially with respect to v , so K -forest now has significantly smaller time complexity than K -tree. The 2-tree wins against 2-forest 10 to 3, where running 2-tree on 2 data sets is impractical. On the other hand, 3-forest wins against 3-tree 7 to 3, where running 3-tree on 5 data sets is impractical; and 4-forest wins against 4-tree 5 to 1, where running 4-tree on 9 data sets is impractical.

C. K -Forest Versus Random Forest

In this section, we compare the performance of our proposed random forest algorithm (K -forest) with random forest proper in terms of generalization error and time complexity. We run our proposed algorithm for $k = 1, 2, 3, 4$, $B = 100$. We also run the random forest proper with the same random forest size ($B = 100$).

Table III shows the average and standard deviations of the error rates of committee machines generated using random forest and K -forest with $k = 1, 2, 3, 4$, $B = 100$. If the difference between random forest and K -forest is statistically significant, we show in

TABLE III

AVERAGE AND STANDARD DEVIATIONS OF THE ERROR RATES OF COMMITTEE MACHINES GENERATED USING RANDOM FOREST WITH $B = 100$ AND K -FOREST WITH $k = 1, 2, 3, 4$, $B = 100$

Set	R-Forest	1-Forest	2-Forest	3-Forest	4-Forest
ACC	8.7 ± 0.8	8.0 ± 0.7	8.3 ± 1.1	8.5 ± 1.1	8.1 ± 0.9
ART	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
BAL	16.3 ± 2.2	13.8 ± 2.4	15.7 ± 3.7	15.4 ± 2.7	13.6 ± 3.4
CAR	8.3 ± 1.7	1.9 ± 0.9	1.4 ± 1.1	1.0 ± 0.7	1.9 ± 0.8
DON	4.8 ± 0.8	4.6 ± 0.6	4.7 ± 0.8	4.6 ± 0.8	4.6 ± 0.7
FLA	38.6 ± 12.3	32.7 ± 14.0	34.7 ± 12.2	37.4 ± 11.3	31.6 ± 9.7
HAY	21.8 ± 7.0	19.5 ± 9.9	21.9 ± 9.4	20.7 ± 7.6	20.0 ± 5.7
KRV	0.6 ± 0.5	0.4 ± 0.5	0.7 ± 0.6	0.4 ± 0.3	0.6 ± 0.4
MON	0.0 ± 0.0	0.0 ± 0.0	0.2 ± 0.8	0.0 ± 0.0	0.0 ± 0.0
MUS	0.0 ± 0.0	0.0 ± 0.1	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
NUR	1.8 ± 0.4	0.3 ± 0.3	0.1 ± 0.1	0.0 ± 0.1	0.0 ± 0.1
POS	28.3 ± 4.7	30.1 ± 9.4	30.3 ± 8.4	35.2 ± 13.5	31.5 ± 8.4
PRO	17.2 ± 8.0	14.0 ± 10.1	5.7 ± 9.5	8.5 ± 10.7	15.5 ± 12.7
SPE	20.2 ± 4.2	17.5 ± 6.1	18.7 ± 5.4	20.9 ± 4.7	21.3 ± 5.8
SPL	4.1 ± 1.0	3.9 ± 1.3	3.8 ± 1.5	4.3 ± 1.4	4.1 ± 1.4
TIC	4.7 ± 1.7	1.4 ± 1.1	1.4 ± 0.7	1.3 ± 1.1	1.3 ± 1.1
VOT	4.4 ± 2.6	4.8 ± 3.3	3.2 ± 2.4	3.4 ± 2.9	3.0 ± 2.4

boldface if K -forest is better, and italic if random forest is better. We see from the results that, K -forest is significantly better than random forest in terms of error rate. 1-forest and 2-forest are significantly better than random forest in 5 and 4 data sets, respectively, 3-forest and 4-forest are significantly better than random forest in 3 and 6 data sets, respectively. In particular, on *tictactoe*, *car* and *nursery* data sets, K -forest has 3, 4, 18 times better error rate compared with random forest. On *monks*, *artificial*, and *mushroom* data sets, all K -forests extract the hidden rule that returns us zero error rate.

On *car*, *nursery*, *promoters*, *tictactoe*, and *vote* data sets increasing k from 1 also improves accuracy. On *artificial*, *spect*, and *vote* data sets, the number of distinct values of the attributes is 2, therefore, the number of distinct possible splits for the extracted features is significantly smaller than other data sets. For this reason, K -forests perform worse on *artificial*, *spect*, and *vote* than on other data sets.

As we have explained in Section V-A, we tune the hyperparameter random attribute size (f) for both random forest and K -forest with values from the set $\{2^0, 2^1, \dots, 2^6, 2^7\}$. Table IV shows the median of the selected optimal f values in tenfolds. In the expanded space, as in *acceptors*, *car*, *krvsnp*, and *nursery*, one has much more possibilities to select from, so K -forest explores a bigger subset of features than random forest. On the other hand, if the underlying problem is easy, as in *artificial*, *monks*, and *mushroom*, a smaller subset of features is enough for K -forest to solve the problem in expanded space.

TABLE IV
MEDIAN OF THE OPTIMAL f VALUES FOR RANDOM FOREST WITH
 $B = 100$ AND K -FOREST WITH $k = 1, 2, 3, 4, B = 100$

Set	R-Forest	1-Forest	2-Forest	3-Forest	4-Forest
ACC	32	128	64	64	128
ART	2	2	1	1	1
BAL	1	1	2	2	4
CAR	4	8	64	64	128
DON	4	8	8	8	16
FLA	8	8	8	16	32
HAY	2	2	2	8	8
KRV	32	128	32	16	32
MON	4	2	2	1	1
MUS	2	2	1	1	1
NUR	4	16	64	32	128
POS	1	1	1	1	2
PRO	8	4	4	8	4
SPE	2	2	4	2	1
SPL	16	8	8	8	8
TIC	8	8	8	8	8
VOT	2	4	2	2	1

TABLE V
AVERAGE AND STANDARD DEVIATIONS OF THE ERROR RATES OF
COMMITTEE MACHINES GENERATED USING RANDOM FOREST
 $B = 500$ AND K -FOREST WITH $k = 1, 2, 3, 4, B = 500$

Set	R-Forest	1-Forest	2-Forest	3-Forest	4-Forest
ACC	8.6 ± 0.9	7.7 ± 0.6	7.9 ± 0.9	7.8 ± 0.7	8.1 ± 0.8
ART	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
BAL	14.9 ± 1.8	13.4 ± 1.9	15.4 ± 3.6	13.5 ± 2.7	13.3 ± 3.2
CAR	8.6 ± 1.5	2.5 ± 1.4	1.3 ± 0.8	1.2 ± 1.0	1.2 ± 1.0
DON	4.5 ± 0.6	4.5 ± 0.5	4.5 ± 0.9	4.7 ± 0.6	4.6 ± 0.9
FLA	37.6 ± 9.7	36.2 ± 12.4	32.0 ± 10.7	32.3 ± 7.1	32.9 ± 12.8
HAY	24.4 ± 7.4	21.4 ± 7.8	17.0 ± 8.2	22.6 ± 7.1	20.4 ± 9.7
KRV	0.6 ± 0.4	0.7 ± 0.4	0.5 ± 0.4	0.6 ± 0.4	0.7 ± 0.5
MON	0.2 ± 0.8	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
MUS	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
NUR	1.8 ± 0.5	0.3 ± 0.2	0.1 ± 0.1	0.0 ± 0.1	0.0 ± 0.1
POS	28.3 ± 4.7	28.3 ± 4.7	32.1 ± 11.3	36.4 ± 10.0	30.3 ± 8.4
PRO	10.2 ± 7.7	12.0 ± 9.6	8.5 ± 8.3	8.5 ± 8.3	13.3 ± 10.7
SPE	16.8 ± 5.6	21.7 ± 2.7	19.4 ± 4.9	20.9 ± 5.4	21.7 ± 5.1
SPL	3.8 ± 1.3	3.3 ± 1.0	3.6 ± 1.0	3.7 ± 1.4	3.6 ± 1.0
TIC	5.1 ± 2.4	1.6 ± 1.0	1.7 ± 0.9	1.5 ± 1.9	1.0 ± 1.1
VOT	3.7 ± 2.4	3.5 ± 2.5	3.9 ± 2.4	3.4 ± 2.7	3.4 ± 2.9

If we compare committee machines generated using random forest and K -forest in terms of the learning time, we see that, almost on all data sets, random forest is significantly better than K -forest. There are two main reasons for this result. First, although the time complexities of random forest and K -forest are equal in theory (in terms of Big-Oh notation, see Section IV for the discussion), constant factor in K -forest is larger than random forest. Second, the learning time of the algorithms also include the hyperparameter tuning. For both random forest and K -forest, we tune f with values from the set $\{2^0, 2^1, \dots, 2^6, 2^7\}$, if applicable. For random forest, we select f attributes from d attributes. On many data sets, the number of attributes d is smaller than 10, so we try only three or four possibilities for f in random forest. On the other hand, for K -forest, we select f attributes from the induced discrete space, which almost always contain much more attributes than $2^7 = 128$, so we always try eight possibilities for f in K -forest. In summary, we try more possibilities for hyperparameter tuning in K -forest than in random forest, where on each trial random forest is more faster than K -forest with a constant factor.

D. Effect of Forest Size (B)

Table V shows the average and standard deviations of the error rates of committee machines generated using random forest with $B = 500$ and K -forest with $k = 1, 2, 3, 4, B = 500$. Again, in this case, we tune the hyperparameter random attribute

TABLE VI
AVERAGE AND STANDARD DEVIATIONS OF THE ERROR RATES OF
COMMITTEE MACHINES GENERATED USING K -FOREST WITH
 $k = 1, 2, 3, 4$ AND $f = 5, 10, 50, 100$

Set	f	1-Forest	2-Forest	3-Forest	4-Forest
ACC	5	16.9 ± 0.5	16.4 ± 0.6	16.4 ± 0.8	16.0 ± 0.6
	10	13.2 ± 0.8	12.5 ± 1.1	13.0 ± 1.4	13.0 ± 1.0
	50	8.0 ± 0.6	8.2 ± 0.4	8.5 ± 0.5	8.2 ± 0.9
	100	7.8 ± 0.9	7.8 ± 1.0	7.9 ± 0.9	8.2 ± 0.7
BAL	5	17.5 ± 4.0	16.8 ± 4.2	14.7 ± 4.0	14.9 ± 2.8
	10	19.2 ± 4.9	16.7 ± 3.7	16.0 ± 5.2	14.6 ± 3.0
	50	19.2 ± 4.6	16.3 ± 3.3	15.9 ± 3.9	15.4 ± 2.5
	100	18.9 ± 4.8	16.7 ± 3.3	16.8 ± 4.9	16.6 ± 4.0
CAR	5	2.0 ± 1.2	1.4 ± 1.1	1.7 ± 1.2	1.8 ± 1.4
	10	1.9 ± 1.1	1.3 ± 1.2	1.6 ± 1.5	1.7 ± 1.4
	50	1.7 ± 0.9	0.8 ± 0.8	0.8 ± 0.9	1.2 ± 0.9
	100	1.4 ± 0.7	0.7 ± 0.9	0.8 ± 0.9	1.0 ± 0.8
DON	5	4.5 ± 0.7	4.4 ± 0.8	4.5 ± 0.6	4.5 ± 0.6
	10	4.6 ± 0.7	4.6 ± 0.8	4.6 ± 0.7	4.6 ± 0.8
	50	4.8 ± 0.7	4.7 ± 0.8	4.6 ± 0.8	4.7 ± 0.7
	100	4.7 ± 0.8	4.6 ± 0.7	4.6 ± 0.7	4.7 ± 0.8
FLA	5	31.8 ± 6.1	31.3 ± 7.1	32.4 ± 8.1	30.1 ± 8.0
	10	34.1 ± 9.9	33.3 ± 9.6	33.0 ± 8.3	33.6 ± 7.3
	50	32.6 ± 14.4	31.7 ± 11.2	31.2 ± 12.0	30.4 ± 8.9
	100	31.3 ± 15.2	29.5 ± 12.0	34.0 ± 11.9	32.8 ± 14.6
HAY	5	16.9 ± 7.2	17.5 ± 6.3	16.2 ± 6.7	18.9 ± 7.4
	10	17.5 ± 6.3	17.6 ± 7.1	17.5 ± 6.3	18.2 ± 7.4
	50	17.6 ± 7.1	16.9 ± 7.1	17.6 ± 7.4	18.9 ± 8.5
	100	17.6 ± 7.1	16.3 ± 7.3	18.7 ± 7.4	18.2 ± 8.8
KRV	5	1.7 ± 0.5	0.8 ± 0.4	0.6 ± 0.3	0.6 ± 0.2
	10	0.7 ± 0.3	0.6 ± 0.4	0.4 ± 0.3	0.5 ± 0.3
	50	0.5 ± 0.5	0.4 ± 0.5	0.4 ± 0.5	0.5 ± 0.5
	100	0.3 ± 0.5	0.4 ± 0.4	0.5 ± 0.5	0.4 ± 0.4
NUR	5	0.3 ± 0.3	0.2 ± 0.1	0.2 ± 0.1	0.2 ± 0.2
	10	0.2 ± 0.2	0.1 ± 0.1	0.1 ± 0.1	0.1 ± 0.1
	50	0.2 ± 0.1	0.0 ± 0.1	0.1 ± 0.1	0.0 ± 0.0
	100	0.2 ± 0.2	0.0 ± 0.1	0.0 ± 0.0	0.0 ± 0.0
POS	5	34.9 ± 10.5	40.6 ± 16.6	38.8 ± 13.5	36.8 ± 12.8
	10	36.8 ± 13.5	37.8 ± 13.7	35.9 ± 12.4	36.5 ± 14.3
	50	38.1 ± 13.4	37.8 ± 12.6	36.8 ± 13.5	37.2 ± 13.3
	100	38.1 ± 15.3	36.8 ± 13.5	36.9 ± 12.3	34.7 ± 12.8
PRO	5	9.5 ± 10.2	11.3 ± 10.9	9.7 ± 12.2	7.7 ± 10.3
	10	9.5 ± 10.2	10.7 ± 12.6	10.5 ± 11.1	10.5 ± 9.7
	50	10.5 ± 7.5	7.8 ± 10.3	8.5 ± 7.3	5.8 ± 6.9
	100	10.2 ± 9.8	12.3 ± 9.4	11.3 ± 10.2	9.7 ± 9.5
SPE	5	19.8 ± 4.7	20.9 ± 5.5	20.9 ± 4.7	21.6 ± 5.8
	10	20.6 ± 4.8	20.9 ± 4.7	19.8 ± 5.8	20.9 ± 5.7
	50	20.9 ± 6.1	20.2 ± 4.5	20.5 ± 6.5	20.5 ± 6.3
	100	20.5 ± 6.6	19.8 ± 5.6	20.9 ± 5.7	20.1 ± 6.2
SPL	5	3.9 ± 1.0	3.8 ± 1.1	3.9 ± 1.0	3.8 ± 1.3
	10	3.4 ± 1.1	3.8 ± 1.3	3.8 ± 1.1	3.5 ± 1.1
	50	4.3 ± 1.4	4.4 ± 1.4	4.3 ± 1.3	4.5 ± 1.3
	100	4.5 ± 1.4	4.4 ± 1.5	4.4 ± 1.5	4.5 ± 1.5
TIC	5	1.4 ± 1.5	0.8 ± 0.8	1.0 ± 0.7	0.8 ± 0.8
	10	0.8 ± 0.8	0.9 ± 0.8	1.1 ± 0.9	0.9 ± 0.9
	50	1.0 ± 1.0	1.7 ± 1.0	1.4 ± 0.7	1.3 ± 0.8
	100	1.0 ± 0.9	1.1 ± 1.2	1.6 ± 0.9	1.3 ± 0.7
VOT	5	3.2 ± 1.9	3.2 ± 2.4	3.4 ± 2.9	2.7 ± 2.6
	10	2.7 ± 2.3	3.2 ± 2.9	2.7 ± 2.3	3.4 ± 2.5
	50	3.6 ± 3.7	3.2 ± 2.7	3.0 ± 2.2	3.4 ± 2.5
	100	3.7 ± 3.6	3.7 ± 2.7	3.2 ± 2.7	3.2 ± 2.7

size (f) for both random forest and K -forest. We see from the results that, if we increase the number of trees, K -forest is again significantly better than random forest in terms of error rate. 1-forest, 2-forest, and 3-forest are significantly better than random forest in 5 data sets, 4-forest is significantly better than random forest in 3 data sets. Again, on *car*, *tictactoe*, and *nursery* data sets, the difference between K -forest and random forest is much more significant.

E. Effect of Random Attribute Size (f)

In Sections V-C and D, we tune the hyperparameter f , random attribute size with different values. In this section, we will look more closely how the performance of K -forest changes with respect

to different f values. For this purpose, on each data set, we run K -forest algorithm with f values from the set $\{5, 10, 50, 100\}$.

Table VI shows the average and standard deviations of the error rates of committee machines generated using K -forest with $k = 1, 2, 3, 4$. For *acceptors*, *car*, *krvsnp*, and *nursery* data sets, increasing number of random attributes improves the performance of K -forest significantly. For these data sets, either the attributes arities are large (on *acceptors*, all features have 4 distinct values, on *car* and *nursery*, average arity is 4), or the number of base attributes is large making the size of the induced discrete space considerably large. As we have mentioned, when the number of attributes is large, but the fraction of relevant attributes is small, random forests perform poorly with small f . Therefore, for these data sets, when we increase f , the performance increases.

On *promoters*, *splice*, and *tictactoe* increasing f from 2 to 5 only increases the performance, but further increase does not change the performance. For these data sets, 5 represents enough percentage of relevant attributes (in the induced discrete space not in the original space), therefore, further increase does not add more.

VI. CONCLUSION

In this brief, we improve the framework [1] that order a subset of k discrete attributes. Using all orderings of values of those k attributes as new extracted features, we propose an extension of random forests, namely, K -forests. Our simulation results on 17 discrete data sets show that our proposed algorithm performs better than their counterpart in terms of error rate.

APPENDIX

k -FOLD PAIRED t TEST

In k -fold paired t test, we test if the error rates of two algorithms, y_{1j} and y_{2j} , $j = 1, \dots, k$, come from the same population.

We calculate the difference at each fold as $d_j = y_{1j} - y_{2j}$ and test if the population of d_j has a mean of zero. The distribution of paired differences d_j is normal since the y_{1j} and y_{2j} distributions are normal. We calculate

$$t'_k = \sqrt{k} \frac{\bar{d}}{s}$$

which, under the null hypothesis, is t distributed with $k - 1$ degrees of freedom. $\bar{d}$ and s^2 are the mean and variance of the paired differences. We reject the null hypothesis that the means are equal if $|t'| > t_{\alpha/2, k-1}$.

REFERENCES

- [1] O. T. Yıldız, "Feature extraction from discrete attributes," in *Proc. 20th Int. Conf. Pattern Recognit.*, Istanbul, Turkey, Aug. 2010, pp. 3915–3918.
- [2] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [3] K. Bache and M. Lichman. (2013). UCI machine learning repository. School Inf. Comput. Sci., Univ. California, Irvine, CA, USA. [Online]. Available: <http://archive.ics.uci.edu/ml>
- [4] J. R. Quinlan, *C4.5: Programs for Machine Learning*. San Mateo, CA, USA: Morgan Kaufmann, 1993.
- [5] J. R. Quinlan, "Induction of decision trees," *Mach. Learn.*, vol. 1, no. 1, pp. 81–106, 1986.
- [6] R. L. De Mántaras, "A distance-based attribute selection measure for decision tree induction," *Mach. Learn.*, vol. 6, no. 1, pp. 81–92, 1991.
- [7] U. M. Fayyad and K. B. Irani, "The attribute selection problem in decision tree generation," in *Proc. 10th Nat. Conf. Artif. Intell.*, San Jose, CA, USA, Jul. 1992, pp. 104–110.
- [8] E. Alpaydm, *Introduction to Machine Learning*. Cambridge, MA, USA: MIT Press, 2010.
- [9] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Hoboken, NJ, USA: Wiley, 1984.
- [10] L. Breiman, "Bagging predictors," *Mach. Learn.*, vol. 24, no. 2, pp. 123–140, 1996.
- [11] A. C. Rencher, *Methods of Multivariate Analysis*. Hoboken, NJ, USA: Wiley, 1995.